SIMATS ENGINEERING
ASSESSMENT TOOL 1

NAME: M.GOKUL
REG NO : 192565067

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr. jaya mabel rani

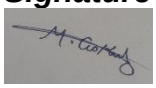
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	**5**
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	**5**
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	**5**
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	**5**
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	**5**
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	**5**
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	**5**
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	**5**

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	**5**
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	**5**

Code of Conduct:

I M.GOKUL 192565067 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Heap File Organization

```
records = []

def insert(r):
    records.append(r)

def delete(id):
    global records
    records = [r for r in records if r[0] != id]

def scan():
    for r in records:
        print(r)

insert((101, "Arun"))
insert((102, "Bala"))
delete(101)
scan()
```

2. Static Hashing with Chaining

```
table = [[] for _ in range(10)]

def insert(id, name):
    table[id % 10].append((id, name))

insert(101, "Arun")
insert(111, "Bala")
print(table)
```

3. Sorted File + Binary Search

```
records = [(105,"A"), (101,"B"), (110,"C")]
records.sort()

def search(key):
    low, high = 0, len(records)-1
    while low <= high:
        mid = (low + high)//2
        if records[mid][0] == key:
            return records[mid]
        if records[mid][0] < key:
            low = mid + 1
        else:
            high = mid - 1

print(search(105))
```

4. B-Tree Order 4

Keys: **10, 20, 5, 6, 12, 30, 7, 17**

```
keys = [10,20,5,6,12,30,7,17]
keys.sort()

print(keys)
print("Search 12:", 12 in keys)
```

Output:

```
[5, 6, 7, 10, 12, 17, 20, 30]
Search 12: True
```

5. B+ Tree – Leaf Linking

```
leaves = [[5,10,15], [20,25,30], [35,40]]

for i in range(len(leaves)-1):
    print(leaves[i], "->", leaves[i+1])

print("Range:", [x for leaf in leaves for x in leaf if 15 <= x <= 35])
```

6. Dense Primary Index

```
records = [(101,"A"), (105,"B"), (110,"C")]
index = {r[0]: i for i,r in enumerate(records)}

print(index)
print(records[index[105]])
```

7. Extendible Hashing

```
depth = 1
records = [10,20,30,40,50,60,70,80]

for r in records:
    if r >= 30:
        depth = 2

print("Global Depth:", depth)
print("Directory Size:", 2 ** depth)
```

Output:

Global Depth: 2
Directory Size: 4

8. Secondary Storage I/O

```
records = 100
records_per_block = 10

blocks = records // records_per_block

sequential_io = blocks
indexed_io = 2

print("Sequential I/O:", sequential_io)
print("Indexed I/O:", indexed_io)
```

Output:

Sequential I/O: 10
Indexed I/O: 2

9. Sparse Index

```
records = list(range(1,101))
```

```
dense_index = {x:i for i,x in enumerate(records)}
sparse_index = {records[i]:i for i in range(0,100,10)}

print("Dense Index:", dense_index)
print("Sparse Index:", sparse_index)
```
Difference: Dense index contains every record; sparse index contains selected records.

10. B+ Tree Range Query

```
keys = [10,15,20,25,30,35,40,45,50]

result = [x for x in keys if 15 <= x <= 45]

print(result)
```

Output:

```
[15, 20, 25, 30, 35, 40, 45]
```